

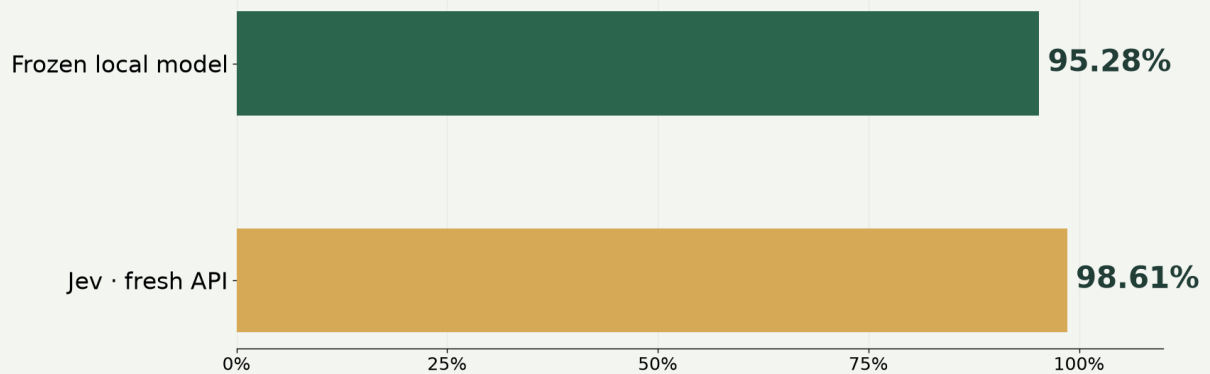
Can Astra Build Its Own Jev?

FILMING GUIDE / 21 SEPTEMBER 2026

The photo-aware local agency, the frozen text challenger, and the measured results.

The stronger model still lost.

360 fresh scenarios · 1,440 policy decisions per engine · zero request failures



Local minus Jev: -3.33 percentage points | 95% interval: -4.38 to -2.29

Synthetic, agent-reviewed references. Two writer-assisted adjudications; no human validation.

Source: round1-deeper-fp16 paired report · frozen before generation · all prespecified cases retained

V2 was not promoted.

The fresh synthetic travel result was 95.28% local versus 98.61% Jev. The app still serves historical V1. Reference repairs and two writer-assisted adjudications are disclosed; no human validation.

1,300 spoken words. 8.4 minutes at 155 WPM, with a planned 10.6-minute runtime including visual holds. Script-only narration source is supplied; no TTS requested.

0:00 - 1:05 / Watch the photo change it

PICTURE

Scene `#demo` at localhost:8770. Run live with stairs, remove photo, run again. Wait for actual results. Open full agency for the twelve-character view.

SAY

Watch Maya, one of our test travellers. She wants her money back if she cancels, and she doesn't want entrance stairs. The written terms work for her. Now I give the system this photo. Her card turns red because it sees the stairs. I take that photo away and run it again. Now it asks us to check. It hasn't proved the stairs disappeared. It has less evidence. This whole thing runs on my computer. I built a Jev-like decision system that uses words and pictures. By the end, you'll understand the pieces, and you'll have my code as a starting point for your own job. You don't need to know machine learning to follow this. Let's get into it.

ON-SCREEN COPY

Watch the photo change it. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

output/qa/vision-causal-checks.json; live V1 plus pretrained vision. Fictional profiles and images.

1:05 - 1:45 / Three parts, one decision

PICTURE

Scene `#map` at localhost:8770. Point to text, image, rules in that order. Keep the whole map visible.

SAY

The big picture is three pieces. One model reads the hotel terms. Another looks at the photos. Then the app checks those answers against what each person wants and what they can afford. That last piece is ordinary code. I trained the text reader for the travel task. I connected an existing image model through OpenJev. I didn't train both models from scratch. What I built is the system around them, so those answers can actually do something useful.

ON-SCREEN COPY

Three parts, one decision. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

travel_lab/vision.py, serve.py, catalogue.py; model selection and source pins.

1:45 - 2:30 / Give it three choices

PICTURE

Scene `#labels` at localhost:8770. Click a wrong label, then Hotel credit. This is a teaching exercise, not an inference request.

SAY

Let's start with the smallest idea. This hotel says you get credit for a future stay. Can Maya get her money back? We don't need a paragraph here. We need to pick from a few answers. Cash back, hotel credit, or we can't tell. In this example, it's hotel credit. A model that picks labels is called a classifier. You've already understood the job before we've touched any machine learning. And that third choice matters. If the policy doesn't say, we want the system to admit that.

ON-SCREEN COPY

Give it three choices. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

Illustrative clause, directly inspectable meaning; the demo text model uses meets/violates/insufficient_evidence.

2:30 - 3:10 / Start with a reader

PICTURE

Scene `#starting` at localhost:8770. Reveal the book visual before naming ModernBERT. No architecture lecture.

SAY

I didn't begin with a blank model. I started with an open model that had already learned patterns in language. Think of it as a reader that already knows something about words, but still needs practice on our particular job. The one in the demo is based on ModernBERT. You don't need to remember the name yet. The useful idea is that someone has done a lot of the starting work, and we can take that starting point and train it further on a narrower task.

ON-SCREEN COPY

Start with a reader. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

models/selection.json pins MoritzLaurer/ModernBERT-base-zeroshot-v2.0. Pretrained model already includes NLI training.

3:10 - 3:55 / Teach it the job

PICTURE

Scene `#training` at localhost:8770. Let the three labeled examples appear over 18 seconds. Point out the third unanswered case.

SAY

So what does that practice look like? We need examples and the answers we want it to learn. Here's a policy that returns money to your card. Here's another that only gives hotel credit. Here's one that never tells us the refund rule. We pair each example with the right label. For this build, we made sample travel policies so we could control the rules. They aren't real hotel listings, and that limits what the test can tell us. Good examples are work. You still have to check them.

ON-SCREEN COPY

Teach it the job. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

data/; synthetic scenario generation and reference audit; conceptual labels simplify the actual NLI training format.

3:55 - 4:40 / Practice, check, adjust

PICTURE

Scene `#practice` at localhost:8770. Use P to hold each feedback phase if needed; R restarts the complete sequence.

SAY

The model makes a guess. Training compares that guess with the label. Then it adjusts some of the model's saved settings so the next guess can be better. We do that over lots of practice examples. That's the basic idea behind fine-tuning. We are giving an existing model more practice on one job. Writing a longer prompt doesn't make the same change to its saved settings. And practice can go wrong. If our examples are too repetitive, the model can learn shortcuts that fall apart when the wording changes.

ON-SCREEN COPY

Practice, check, adjust. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

travel_lab training code and runs/training.jsonl. Diagram is a simplified teaching model, not a visualization of individual weights.

4:40 - 5:25 / Keep the exam sealed

PICTURE

Scene `#test` at localhost:8770. Practice cards left, sealed final envelope right. Explain the boundary before moving to scores.

SAY

Now imagine giving someone a test after showing them all the answers. A high score wouldn't mean very much. So we keep some examples away from training. We use a separate practice check while we're making changes, then freeze the final model before opening the final exam. For the later challenger, both it and Jev got the same new travel scenarios. We also kept the mistakes. If I look at the final test, change the model, and call that same test fresh, I've broken the point of the exercise.

ON-SCREEN COPY

Keep the exam sealed. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

experiments/v2/freezes/deeper-fp16-20260921.json and final protocol. No final/public tuning.

5:25 - 6:25 / The test kept us honest

PICTURE

Scene `#results` at localhost:8770. Hold both bars on a common zero-to100 scale. Read rounded numbers aloud; exact numbers stay visible. Do not draw improvement arrows between V1 and V2.

SAY

Here's where the test kept me honest. My later challenger agreed with our reference answers about ninety-five percent of the time. Jev was closer to ninety-nine percent on those same cases. So I can't tell you I beat Jev at this task. I didn't. These were made-up travel cases with checked reference answers, not a field test across real hotels. Also, the challenger on this chart is a later experiment. The app you saw still runs the earlier version, which scored sixty-six point five percent on its own older travel test. Different tests, different versions. The working interface doesn't make the model reliable. The value here is that you can inspect what I built, see where it fails, and use the process to test your own idea.

ON-SCREEN COPY

The test kept us honest. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

output/benchmarks/FROZEN-V2-RESULTS.md; V1 summary.json.360 synthetic cases,1440 judgments, no human validation; nonpromotion.

6:25 - 7:10 / Give the system eyes

PICTURE

Scene `#vision` at localhost:8770. Play pixels / observations / app composition. Point back to the same entrance photo from the opening.

SAY

Now let's add the part the text reader can't see. The photos. I connected OpenJev to an existing image model running on this Mac. It gets the actual picture and a few questions about visible features. Is there a swimming pool? Can it see stairs? It returns observations that our app can use. We don't send it a filename that gives away the answer. Using more than one kind of input is what multimodal means here. Words and pictures. This image model was already trained. My work was connecting it to the travel system and testing the decisions it changes.

ON-SCREEN COPY

Give the system eyes. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

experiments/openjev-vision/pin.json; real-image-observations.json; pretrained DiffusionGemma MLX via OpenJev.

7:10 - 7:50 / A picture leaves things out

PICTURE

Scene `#boundary` at localhost:8770. Show Kyoto pond; click conclusion after posing the questions.

SAY

Take this garden. There's water in the photo. That doesn't establish a swimming pool. And even a clear swimming pool wouldn't tell us whether it's free, open today, or included in this booking. The photo answers questions about what's visible. The terms answer questions about what's promised. That's also why removing the stairs photo sends Maya to review. We lost the evidence. We haven't discovered a different entrance. This is a useful place to be conservative.

ON-SCREEN COPY

A picture leaves things out. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

Actual photo observation plus application truth boundaries. Photos generated and shared by destination.

7:50 - 8:30 / The app makes the call

PICTURE

Scene `#rules` at localhost:8770. Animate three gates, then open a traveller drawer in the full app to inspect evidence.

SAY

Once we have the answers, the app applies the traveller's rules. Does the price fit? Do the written terms meet the requirement? Does a photo show something they asked for, or something they want to avoid? A failed requirement keeps the card red. If the evidence is missing, the card asks for review. A match means the checks passed according to the models and our rules. It still isn't a verified booking recommendation. We can click through to see which clause or photograph affected the result.

ON-SCREEN COPY

The app makes the call. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

catalogue.verdict and vision.apply_visual_requirements; statuses and evidence links.

8:30 - 9:05 / Local has a different bill

PICTURE

Scene `#cost` at localhost:8770. Show the two cost columns. No unsourced ratio or free-compute headline.

SAY

Because these checks run locally, I'm not paying a model provider for each local request. But the computer still costs money, it uses power, and setup takes time. The image model is a separate download of about sixteen gigabytes, and this version uses Apple silicon. So I wouldn't promise you it's a fixed fraction of the price of every language model. Compare the total cost on the job you actually have. Small repeatable decisions are a good place to investigate.

ON-SCREEN COPY

Local has a different bill. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

Pinned local runtime and download; no controlled LLM total-cost benchmark performed.

9:05 - 9:55 / Now change the job

PICTURE

Scene `#repurpose` at localhost:8770. Switch Travel / Support / Product photos. These are illustrative adaptation recipes, not live integrations.

SAY

You can take the same process somewhere else. Maybe you want to send support requests to the right team. Start with one clear question and the answers you're willing to accept. Collect examples, label them carefully, and keep some aside to test later. If pictures matter, choose the visual questions as well. Then decide what your app should do with each answer. For a new subject, you will need your own examples and tests. Changing a folder name won't make the travel model an expert in that subject. But you have the structure, the code, and the mistakes I already made to start from.

ON-SCREEN COPY

Now change the job. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

docs adaptation guide; actual travel scope preserved.

9:55 - 10:35 / Start from my work

PICTURE

Scene `#take-it` at localhost:8770. Show private repository during prep. Only say viewers can download after access or a separate public resource has been arranged.

SAY

I've organized the project so you can follow the same path. The agency is the working demo. The explainer is this page. The docs show how the parts fit together and how to run them. And the evidence folder keeps the results, including the losses. You'll also find the source credits for the open models and tools I built on. Start by running the example. Then pick one job you understand well. Change the examples, check the results, and only then decide whether this is useful for you.

ON-SCREEN COPY

Start from my work. Minimal labels in the scene.

EDITING NOTE

Hold the final visual while speaking. P pauses; R resets. Live requests may finish while animation is paused.

SOURCE / TRUTH CARD

Private promptadvisers/away-together; no public access implied. Complete companion artifact and upstream notices.

Before the take

Start the OpenJev vision service on localhost:8081 and the agency on localhost:8765. Reset with R. Select Four friends. Run Atlantic hideaway fresh. P pauses or resumes the presentation. Escape closes details. Warm both models before recording; cold load is separate from warm request timing.

The charts show saved benchmark results. The agency serves V1, while the final challenger result is V2. Photos, travellers and offers are fictional. Source-clause linkage is structured application logic.

Truth boundaries

Final references are synthetic and agent-reviewed, with 52 document repairs and two non-unanimous writer-assisted adjudications. All 360 scenarios remain. No human validation. The public workflow lane measures teacher agreement; upstream exposure may apply. Thirty-five workflow decisions failed the frozen context limit and remain in the denominator. Official TypeSafe benchmark scores were not reproduced.

Measured agency timing

Active V1: cold first HTTP request 10.1 seconds including loading. Three fresh forty-offer scans took 4.0, 4.2 and 4.5 seconds. Warm HTTP median 105 ms, 121 unique fresh requests, zero failures. These are text-only historical timings, separate from V2 model-only latency. The photo-aware 40-offer integration scan took 40.7 seconds in its recorded run.

Companion resource

Show the verified local bundle, including both checkpoints and original audit evidence. Add a public download link only once separately arranged. The OpenJev image branch ran locally; its pinned model is downloaded separately. The earlier text-only diffusion comparison was not executed. No public upload, deployment or messaging is implied.